

TOOLCHAIN CANON

2026-03-18

Dexter Hadley, MD/PhD

Hadley Lab CANONIC

Abstract

Governance drives code. Never the reverse.

hadleylab.org Governed Research. Every claim cited.

Contents

All executables live at `~/.canonic/bin/`. All governance lives at `~/CANONIC/`. The boundary is absolute. Fleet membership is auto-discovered from runtime state. Frontend identity is governed by `HTTP.json`. Surface types by `SURFACE.json`.

See [README.md](#) for tool documentation.

Constraints

MUST: `magic.c` is the single source of dimension truth

MUST: `magic-heal` settles `CANON/README/VOCAB/{SCOPE, ...}`

MUST: `build` validates via `magic scan` before any deployment

MUST: `build-scopes-json` autodiscovers scopes across runtime

MUST: `Service->fleet` mapping is a direct projection

MUST: Canonical projection artifact is `~/.canonic/`

MUST: `build` validates HTTP fleet roots at `MAGIC 255`

MUST: GOV `design_language` is the only lane ontology

MUST: Fleet frontend git remotes use governed ownership

MUST: `deploy` ledgers to `LEDGER`

MUST: Every tool has documented purpose, inputs, outputs

MUST: All executables live at `~/.canonic/bin/` code

MUST: All governance lives at `~/CANONIC/` governance

MUST: Fleet sites are discovered from runtime state

MUST NOT: Deploy without validation passing

MUST NOT: Hardcode paths discover from governance tree

MUST NOT: Duplicate enforcement `magic.c` is the only enforcement

MUST NOT: Add parallel `scope->fleet` mapping layers out

MUST: `ECON.v1.json` and `WALLET.v1.json` govern vault

MUST: Every compiled JSON includes `_generated: {compiles, ...}`

MUST: vault publish injects `_generated` into `econ.json`

MUST: CI/CD enforces `magic validate 255` on push to prod

MUST NOT: Edit `README.md` by hand `magic-heal` regenerates

MUST: Build pipeline emits structured output (PHASES)

MUST: `magic validate` output follows structured format

MUST: `deploy` emits fleet-wide status summary

MUST: Error output identifies the failing contract

MUST NOT: Emit unstructured prose in pipeline output

MUST NOT: Silence failures every gate produces evidence

MUST: Runtime reads `CANONIC` config from compiled TOML

MUST: Worker env vars are REQUIRED in `wrangler.toml`

MUST NOT: Swallow exceptions silently every catch must log

MUST: `validate-compliance` gate runs in build pipeline

MUST: `magic-tag` validates 255 before creating any tags

MUST: `magic-tag` appends to `TAGS.md` never edits existing

MUST: `freeze-*` tags block deploy from pushing into prod

MUST: `deploy` checks freeze state and warns before proceeding

MUST: Tag names follow governed naming convention (semver | freeze-* | patent-filing-*)
MUST: TAG.v1.json governs tag record schema
MUST NOT: Create tags on dirty working tree
MUST NOT: Delete or move existing tags
MUST NOT: Deploy breaking interface changes during freeze without explicit --override-freeze
MUST: Toolchain dispatch uses STRATEGIES registry every contract declares 'compiler:' in header
MUST NOT: Hardcode outputfunction mappings COMPILERS dict is dead
MUST: discover_all() is the single GOV tree walk galaxy + scopes consume it
MUST: classify_kind() reads 'kind:' from CANON.md header no path heuristics
MUST: VALID_KINDS = {ORG, SERVICE, USER, VERTICAL, SCOPE, PRINCIPAL} unknown kind = fail
MUST: build-surfaces parses each scope into IR via parse_scope() emitters consume IR, never re-
MUST: Build phases live at build_phases/[0-9][0-9]-* runner discovers via glob
MUST: Each phase sources fleet.conf, exit 0 = pass, non-zero = build stops
MUST NOT: Inline phase logic in the build runner each phase is a self-contained script
MUST: validate-coin-wiring enforces 5 hard gates: manifest, tier_floor, wallet_existence, shop_r
MUST NOT: Treat COIN violations as warnings compliant or fail
MUST: Every function exists in exactly one file duplicates deleted, not deprecated
MUST NOT: Re-export wrappers or compatibility shims for moved functions
MUST: build-surfaces reads FRESHNESS.json for source dependency rules not hardcoded
MUST: --incremental is default for local builds, --no-cache for CI
MUST: Cache is runtime artifact at path declared in FRESHNESS.json never in GOV tree
MUST NOT: Skip validation after incremental build 255 gate always runs

TOOLCHAIN / CANON / MAGIC